

WAD Chat

LLM Chat Application

Assignment	Web Application Development
Student ID	ISU520196_3
Date	May 2026
Repository	github.com/brosjsy/llm_assignment_ISU520196_3

1. Project Overview

WAD Chat is a ChatGPT-style conversational application that runs a local Large Language Model (LLM) entirely on CPU — no cloud API or GPU required. Users register or sign in via password or GitHub OAuth, create named conversation threads, and exchange messages with the AI in real time through a streaming Server-Sent Events (SSE) interface.

Technology Stack

Layer	Technology	Purpose
Web framework	FastAPI 0.115	Async REST API + HTML routes
Database	PostgreSQL 16	Persistent storage for users, chats, messages
Migrations	Alembic 1.14	Versioned schema changes
Session store	Redis 7	Refresh token sessions (30-day TTL) + message cache
Auth tokens	JWT (python-jose)	Access token 30 min / Refresh token 30 days
Social auth	GitHub OAuth 2.0	Sign-in alongside password login
Password hash	Argon2 (passlib)	Secure credential storage
LLM runtime	llama-cpp-python	Local GGUF model, CPU inference
Templating	Jinja2	Server-rendered HTML views
HTTP client	httpx	Async GitHub API calls in AuthService

Architecture Pattern

The UI is **server-rendered HTML** (Jinja2 templates), so the project follows **MVC — Model · View · Controller**:

- **Models** — SQLAlchemy ORM classes (*User*, *Chat*, *Message*).
- **Views** — Jinja2 HTML templates (login, chat list, chat detail).
- **Controllers** — FastAPI route handlers; every endpoint calls **exactly one** service method. No business logic in the router.

JWT + Redis Session Flow

- On **register / login / GitHub OAuth**: access token (30 min) + refresh token (30 days) issued; `session_id` stored in Redis with 30-day TTL.
- Refresh endpoint validates Redis entry, rotates session (deletes old, creates new).
- Logout deletes the Redis key immediately, invalidating the refresh token server-side.

Bonus Features

- **Streaming output** — tokens stream via SSE; UI renders them live with a blinking cursor.

- **Redis cache** — message history cached per chat for 5 min; invalidated on new message or delete.
- **Auto-title** — first user message (up to 40 chars) becomes the chat title.

2. Code Structure

The project is organised into feature packages under *app/*. Cross-cutting concerns (config, database, Redis) live directly under *app/*.

```
app/  
auth/  
depends.py FastAPI dependency: extract current user from JWT  
router.py Auth controllers – one svc call per endpoint  
schemas.py Pydantic I/O models  
security.py Argon2 hashing, JWT creation/decoding  
service.py All auth logic (register, login, refresh, logout, GitHub OAuth)  
chats/  
models.py SQLAlchemy ORM: Chat, Message  
router.py Chat controllers – one svc call per endpoint  
schemas.py Pydantic I/O models  
service.py Chat CRUD + LLM orchestration (ask, ask_stream)  
llm/  
service.py LLMService: singleton model, generate, generate_streaming, build_prompt  
users/  
models.py SQLAlchemy ORM: User  
config.py Pydantic BaseSettings – reads .env  
database.py Async engine, session factory, DeclarativeBase  
main.py FastAPI app: static files + router registration  
redis_client.py Module-level async Redis client  
templates/  
base.html HTML shell  
auth/login.html Login + Register tabs; GitHub OAuth button  
chats/list.html Chat list sidebar + welcome screen  
chats/detail.html Message thread; SSE streaming via EventSource  
static/css/style.css Dark-theme UI (CSS custom properties)  
alembic/versions/0001_initial.py Creates users, chats, messages tables  
docker-compose.yml PostgreSQL 16 + Redis 7  
requirements.txt Pinned Python dependencies
```

Key Module Descriptions

app/auth/service.py — AuthService

Owns all authentication business logic. Public methods: `register()`, `login()`, `refresh()`, `logout()`, `github_oauth_callback()`, `get_or_create_github_user()`. Internal: `_issue_tokens()` generates the JWT pair and stores the session in Redis.

app/chats/service.py — ChatService

Owns all chat and LLM orchestration. `ask()` is the full non-streaming turn: validate ownership -> auto-title -> save user message -> get cached history -> generate LLM answer -> save assistant message -> return `MessageResponse`. `ask_stream()` is an async generator yielding SSE tokens and persisting on completion.

app/llm/service.py — LLMService

Thin wrapper around llama-cpp-python. Module-level singleton so the model loads once. `generate()` returns a full completion; `generate_streaming()` yields tokens; `build_prompt()` formats the last 6 messages into a User/Assistant dialogue prompt.

app/auth/depends.py — get_current_user()

FastAPI dependency injected into every protected route. Reads JWT from the Authorization: Bearer header first, then falls back to the `access_token` cookie. Returns the User ORM object or raises HTTP 401.

3. API Structure

The API is built with FastAPI and follows REST conventions. Protected endpoints accept the JWT as *Authorization: Bearer <token>* or via the `httponly access_token` cookie. Interactive documentation: **!docs** (Swagger UI).

Authentication Endpoints (prefix: /auth)

Method	Path	Auth	Description
POST	/auth/register	—	Register with login + password. Returns JWT pair. 400 if login
POST	/auth/login	—	Login. Returns JWT pair in body + httponly cookies. 401 if inval
POST	/auth/refresh	—	Rotate refresh token. Body: {refresh_token}. Returns new toke
POST	/auth/logout	JWT	Invalidate session in Redis; clear cookies.
GET	/auth/github	—	Redirect browser to GitHub authorization page.
GET	/auth/github/callback	—	Handle GitHub code exchange; create/find user; redirect to /ch

Chat Endpoints (all require JWT)

Method	Path	Auth	Description
GET	/api/chats	JWT	List all chats for the authenticated user.
POST	/api/chats	JWT	Create a new chat. Body: {title}. Returns 201.
GET	/api/chats/{id}	JWT	Get chat with full message list. 404 if not found.
DELETE	/api/chats/{id}	JWT	Delete chat and messages. Clears Redis cache. 204.
POST	/api/chats/{id}/ask	JWT	Non-streaming LLM turn. Body: {content}. Returns assistant M
GET	/api/chats/{id}/stream	JWT	SSE stream. ?content=... Yields tokens; final: data: [DONE].

HTML Views (cookie JWT, server-rendered)

Method	Path	Auth	Description
GET	/	—	Login/Register page. Redirects to /chats if authenticated.
GET	/chats	JWT	Chat list sidebar + welcome screen.
GET	/chats/{id}	JWT	Chat conversation with SSE streaming input.

Request / Response Schemas

RegisterRequest: { login: str, password: str, email: str | null }

LoginRequest: { login: str, password: str }

RefreshRequest: { refresh_token: str }

TokenResponse: { access_token: str, token_type: 'bearer' }

ChatCreate: { title: str = 'New Chat' }

ChatResponse: { id: int, title: str, created_at: datetime }

ChatDetail: ChatResponse + { messages: [MessageResponse] }

MessageResponse: { id: int, role: str, content: str, created_at: datetime }

AskRequest: { content: str }

4. Database Schema

PostgreSQL is the primary relational database. Schema changes are applied exclusively through **Alembic migrations**. SQLAlchemy async ORM with asyncpg is used for all queries.

Table: users

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Auto-incrementing surrogate key
login	VARCHAR(150)	UNIQUE NOT NULL	Username; unique across system
email	VARCHAR(255)	NULL	Optional email address
hashed_password	VARCHAR(255)	NULL	Argon2 hash; NULL for GitHub-only accounts
github_id	VARCHAR(100)	UNIQUE NULL	GitHub user ID; NULL for password accounts
is_active	BOOLEAN	NOT NULL	Soft-disable flag; default TRUE
created_at	TIMESTAMP	NOT NULL	Account creation time (UTC)

Table: chats

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Auto-incrementing surrogate key
title	VARCHAR(255)	NOT NULL	Chat title; auto-set from first message
user_id	INTEGER	FK users.id CASCADE	Owner; deleting user cascades to chats
created_at	TIMESTAMP	NOT NULL	Thread creation time (UTC)

Table: messages

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Auto-incrementing surrogate key
chat_id	INTEGER	FK chats.id CASCADE	Parent chat; deleting chat cascades
role	VARCHAR(20)	NOT NULL	'user' or 'assistant'
content	TEXT	NOT NULL	Full message text
created_at	TIMESTAMP	NOT NULL	Message timestamp (UTC)

Relationships

User **1** — **N** Chat (a user owns many chats; ON DELETE CASCADE)

Chat **1** — **N** Message (a chat has many messages; ON DELETE CASCADE)

Redis Data (non-relational)

Two key namespaces are used in Redis:

Key pattern	Value	TTL	Purpose
refresh:{session_id}	user_id (str)	30 days	Refresh token session – deleted on logout/r
chat_messages:{chat_id}	JSON array	5 minutes	Cached message list – invalidated on write/

5. User Interface — Screenshots

All screenshots were captured from the live application running locally at **http://localhost:8000**. The UI is built with server-rendered Jinja2 templates and a dark theme using CSS custom properties. No frontend framework is required — minimal vanilla JavaScript handles form submission (Fetch API) and real-time streaming (EventSource SSE API).

Login Page — Login Tab

The login page is served at the root URL (/). It shows a dark card centred on the screen with two tabs. The **Login tab** (shown below) has Username and Password fields, a Sign in button, a divider, and a **Continue with GitHub** OAuth button.

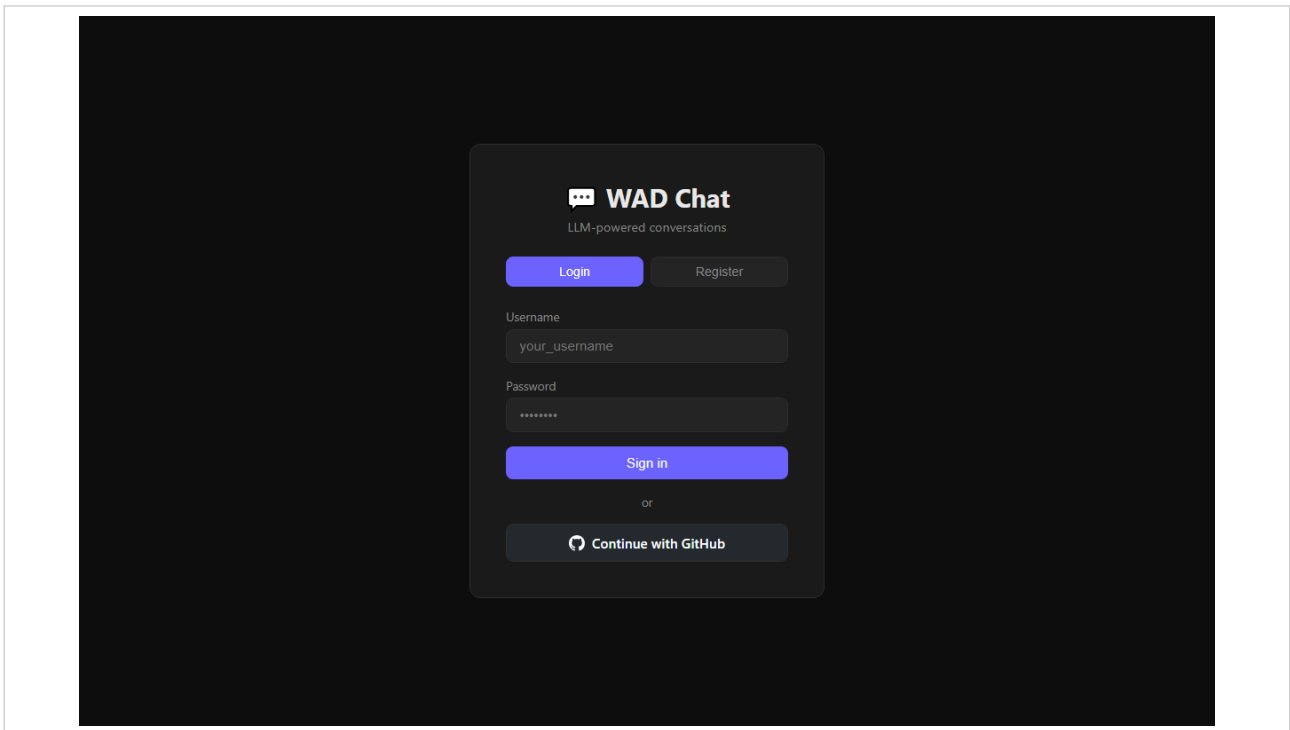


Figure 1 — Login page (Login tab active) with GitHub OAuth button

Register Page — Register Tab

Clicking the **Register tab** reveals an additional optional Email field. On successful registration the server issues JWT tokens, sets httponly cookies, and redirects the browser to `/chats`.

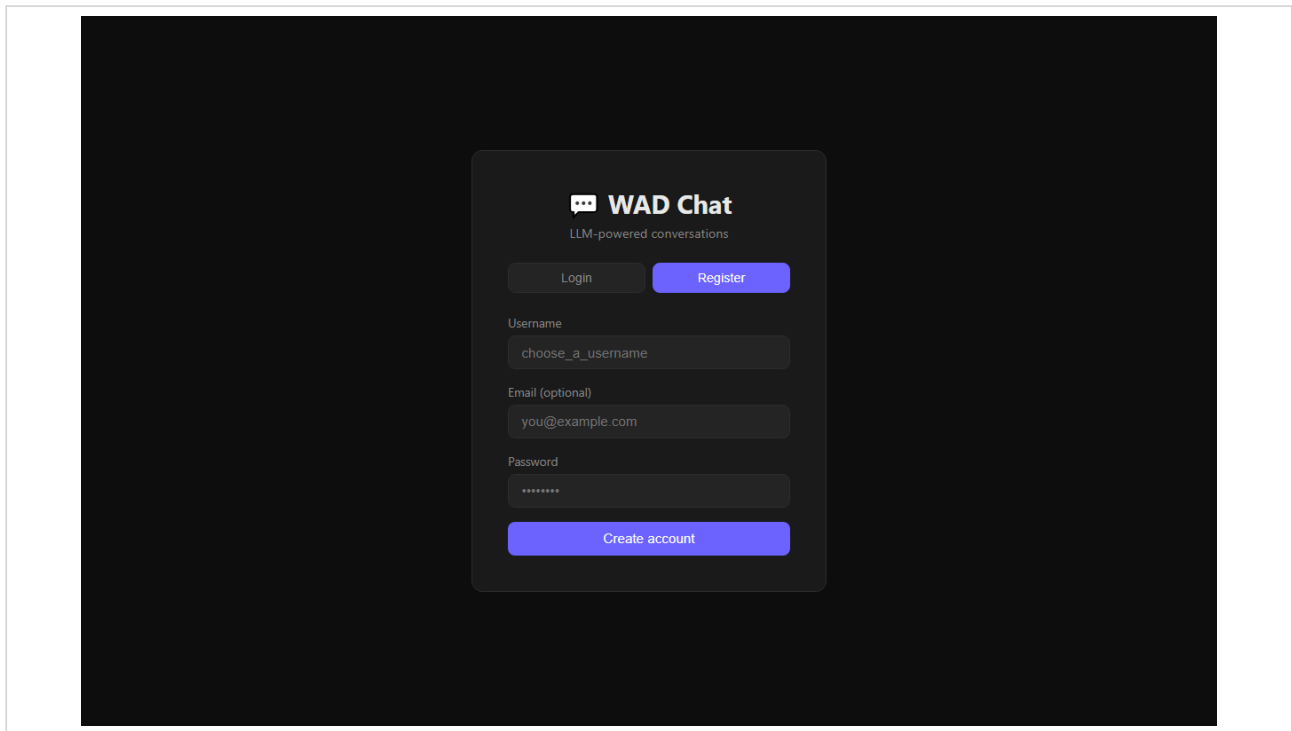


Figure 2 — Register tab with Username, Email (optional) and Password fields

Chat List — Welcome Screen

After authentication the user lands on `/chats`. The two-panel layout shows: a **260 px sidebar** on the left listing all chats with auto-generated titles and a **+ New Chat** button; a main content area with a welcome message when no chat is selected. The sidebar footer shows the logged-in username and a Logout button.

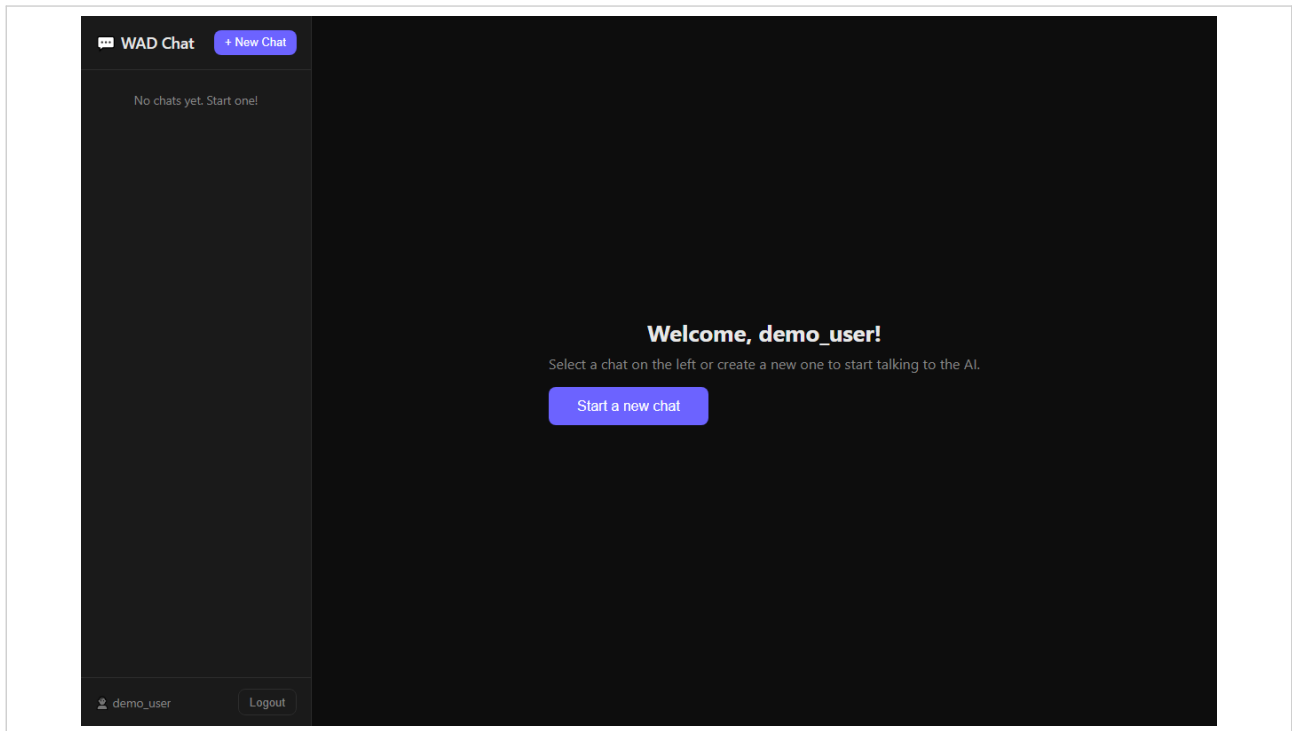


Figure 3 — Chat list welcome screen after login

Chat Conversation — Live Streaming

Inside a chat, user messages appear right-aligned in purple bubbles; assistant responses appear left-aligned in dark bubbles. Tokens stream in real time via SSE (Server-Sent Events) and are appended to the bubble one-by-one with a blinking cursor (█). The chat title is auto-set from the first user message. The sidebar shows both chats with their auto-generated titles.

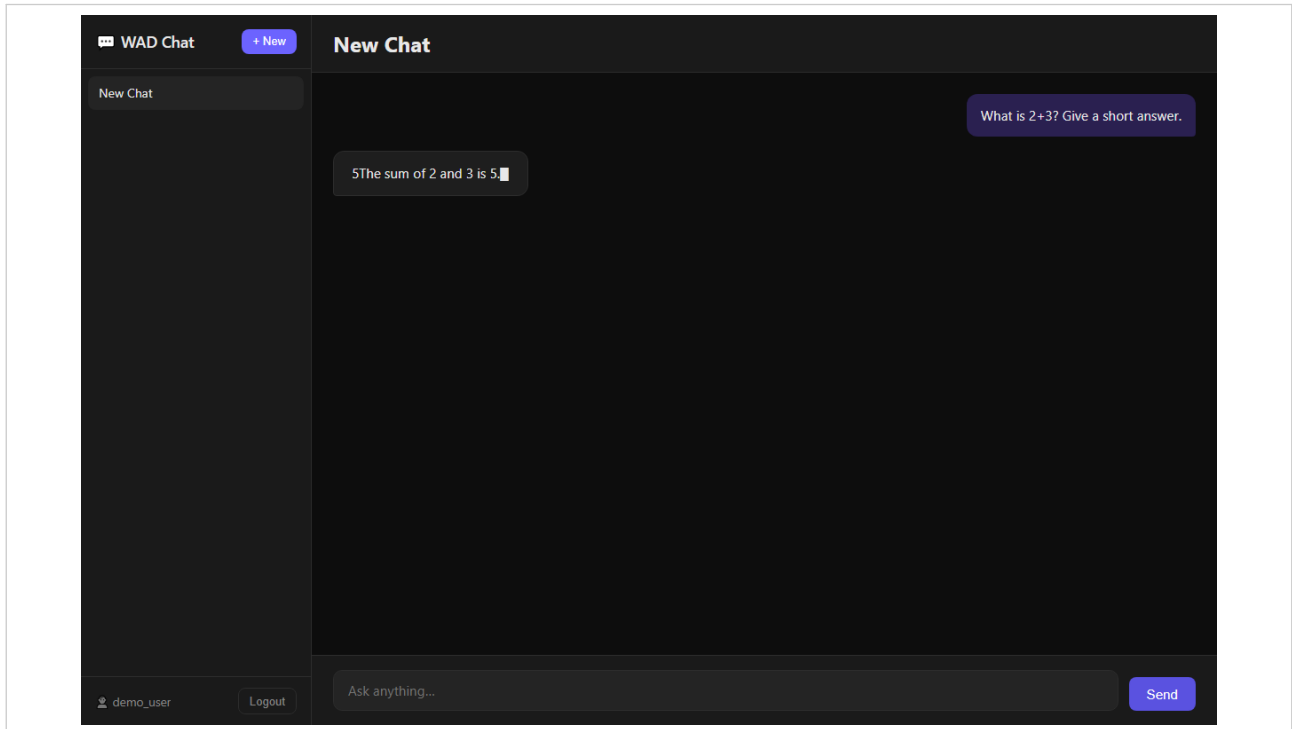


Figure 4 — Active chat with streaming LLM response and auto-titled sidebar entries

Swagger UI — Interactive API Documentation

FastAPI auto-generates interactive documentation at **/docs**. All endpoints are listed and grouped by tag (auth, chats). Each endpoint shows the HTTP method, path, request schema, and response schema. The *Try it out* button allows sending real requests directly from the browser.

auth

POST	/auth/register	Register	▼
POST	/auth/login	Login	▼
POST	/auth/refresh	Refresh Token	▼
POST	/auth/logout	Logout	▼
GET	/auth/github	Github Login	▼
GET	/auth/github/callback	Github Callback	▼

chats

GET	/	Index	▼
GET	/chats	Chats Page	▼
GET	/chats/{chat_id}	Chat Detail Page	▼
GET	/api/chats	Api List Chats	▼

Figure 5 — Swagger UI at /docs showing all API endpoints

6. How to Run the Application

Follow these steps exactly to run WAD Chat on any machine.

Prerequisites

Install the following before starting:

Python 3.11+ – <https://python.org/downloads>
Docker Desktop – <https://docs.docker.com/get-docker/> (provides PostgreSQL + Redis)
Git – <https://git-scm.com>
A **GGUF model file** – any quantised LLaMA/Qwen/Mistral model, e.g. from Hugging Face
A **GitHub OAuth App** (only needed for GitHub login – optional for basic testing)

Step 1 — Clone the repository

Open a terminal and run:

```
git clone https://github.com/brosjsy/llm_assignment_ISU520196_3.git
cd llm_assignment_ISU520196_3
```

Step 2 — Start PostgreSQL and Redis

Docker Desktop must be running first, then:

```
docker-compose up -d
```

Step 3 — Create a virtual environment

Windows:

```
python -m venv venv
venv\Scripts\activate
```

Step 4 — Install Python dependencies

With the virtual environment activated:

```
pip install -r requirements.txt
```

Step 5 — (Windows only) llama-cpp-python

If the install above fails for llama-cpp-python, use the pre-built CPU wheel:

```
pip install llama-cpp-python --extra-index-url
https://abetlen.github.io/llama-cpp-python/whl/cpu
```

Step 6 — Place the GGUF model

Copy your model file to the project root and name it model.gguf:

```
cp /path/to/your/model.gguf ./model.gguf
```

Step 7 — Configure environment variables

Copy the example file and edit it:

```
cp .env.example .env
```

Step 8 — Edit .env

Open `.env` and set the following values:

Variable	Default / Example	Description
DATABASE_URL	postgresql+asyncpg://postgres:postgres@localhost:5432/postgres	Asyncpg SQLAlchemy connection string
REDIS_URL	redis://localhost:6379	Redis connection URL
SECRET_KEY	(change this!)	JWT signing secret — use a long random string in production
ACCESS_TOKEN_EXPIRE_MINUTES	30	Access token lifetime in minutes
REFRESH_TOKEN_EXPIRE_DAYS	30	Refresh token lifetime in days (also the Redis TTL)
GITHUB_CLIENT_ID	your_github_client_id	From GitHub Developer Settings (optional)
GITHUB_CLIENT_SECRET	your_github_client_secret	From GitHub Developer Settings (optional)
GITHUB_REDIRECT_URI	http://localhost:8000/auth/github/callback	Must match GitHub App settings
MODEL_PATH	model.gguf	Relative or absolute path to your GGUF model file

GitHub OAuth App Setup (optional — needed only for GitHub login)

- Go to <https://github.com/settings/developers> and click **New OAuth App**.
- Set **Homepage URL** to: `http://localhost:8000`
- Set **Authorization callback URL** to: `http://localhost:8000/auth/github/callback`
- Copy the **Client ID** and **Client Secret** into `.env`.

Step 9 — Run database migrations

```
alembic upgrade head
```

Step 10 — Start the server

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Open **`http://localhost:8000`** in your browser. Register an account, create a chat, and start talking to the AI.

API documentation (Swagger UI) is available at **`http://localhost:8000/docs`**.

7. Security Design

Password storage

Passwords are hashed with Argon2 (memory-hard, GPU-resistant) via `passlib[argon2]`. Plain-text passwords are never stored, logged, or transmitted after the initial request.

JWT tokens

HS256-signed JWTs. Access tokens expire in 30 minutes; refresh tokens in 30 days. Both are stored in `httponly`, `SameSite=Lax` cookies — inaccessible to JavaScript, preventing XSS-based token theft.

Session invalidation

Refresh token session IDs are stored in Redis with a matching TTL. Logout deletes the Redis key immediately, revoking the refresh token server-side regardless of its JWT expiry. Token rotation on refresh limits the replay window.

GitHub OAuth

Authorization Code flow only. The GitHub client secret stays server-side. The received GitHub access token is used only to fetch the user profile and never stored.

Input validation

All API request bodies are validated by Pydantic v2 before reaching service code. SQLAlchemy ORM with parameterized queries prevents SQL injection.